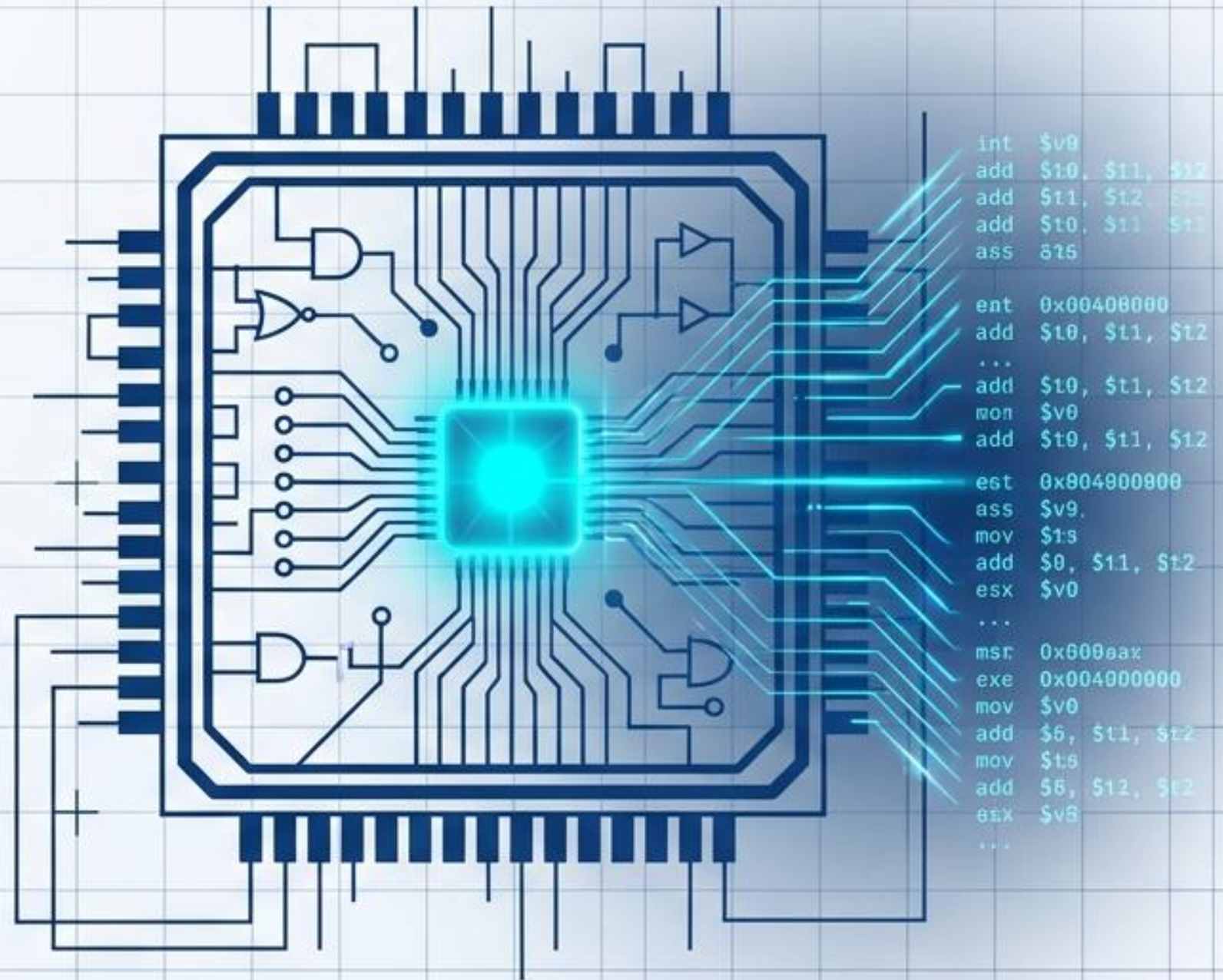


Aula 2

Fundamentos da Arquitetura MIPS e Operação no MARS

Da fundação teórica do hardware
à simulação executável.

Prof. Danilo Monteiro Souza



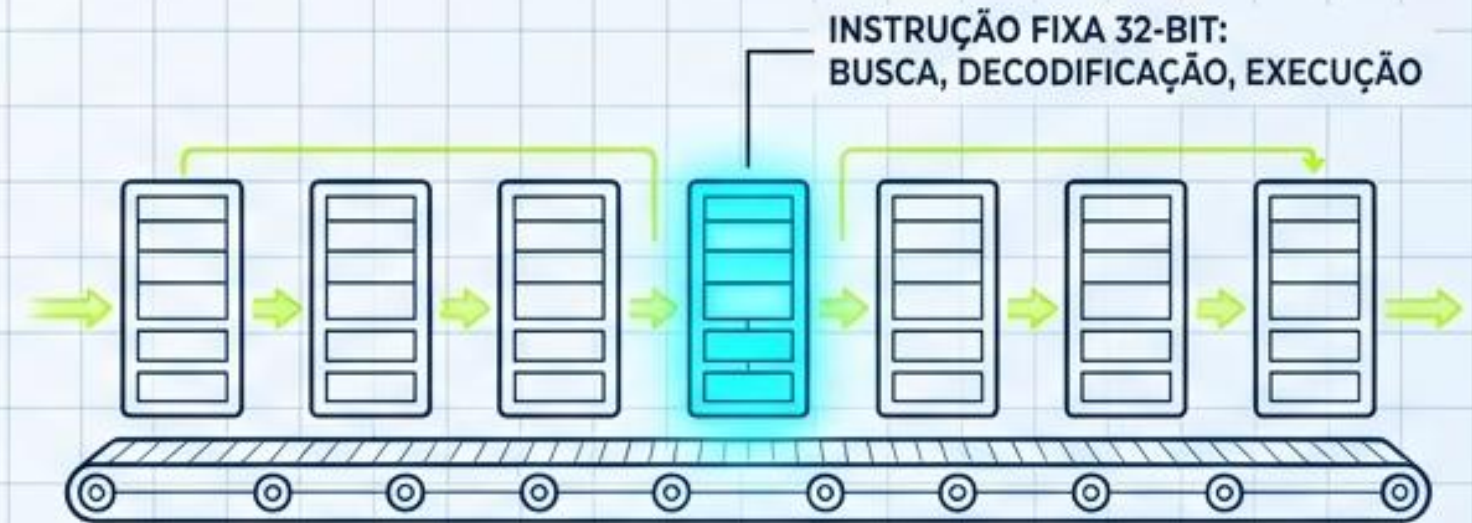
A Filosofia RISC e o Princípio da Simplicidade

A arquitetura MIPS (Microprocessor without Interlocked Pipeline Stages) é o exemplo clássico da **filosofia RISC (Reduced Instruction Set Computer)**. Em oposição a processadores complexos, o MIPS opera sob um princípio de design implacável: **A simplicidade favorece a regularidade.**

CISC (Complexo)



RISC (Simples)



Menos formatos de instrução e simetria rigorosa permitem que o processador decodifique e execute tarefas com extrema velocidade e eficiência.

O Gargalo Físico e a Necessidade de Registradores

A memória principal (RAM) é vasta, porém fisicamente distante do núcleo de processamento, tornando-a lenta para acesso constante. Para alcançar a velocidade do clock, a CPU exige armazenamento interno imediato.

No MIPS, todas as operações aritméticas e lógicas ocorrem estritamente dentro dos registradores.



Matriz Funcional do Banco de Registradores

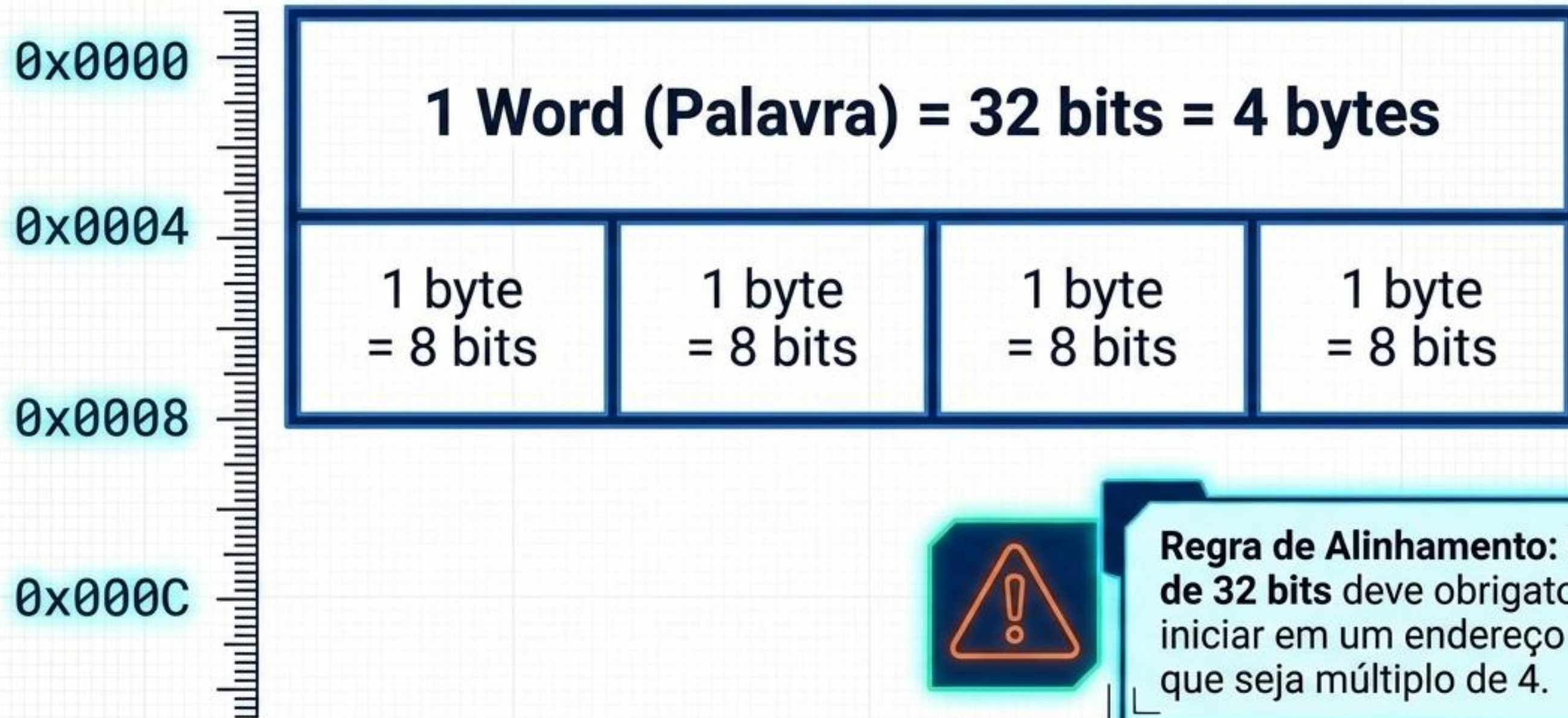
\$zero (0)	Constante Fixa. Retorna sempre o valor 0. Fixo em hardware.
\$t0 - \$t9 (8-15, 24-25)	Temporários Voláteis. Subrotinas podem sobrescrevê-los. Não são preservados.
\$s0 - \$s7 (16-23)	Salvos Seguros. Valores de longa duração. Devem ser restaurados se uma subrotina os utilizar.
\$a0 - \$a3 (4-7) \$v0 - \$v1 (2-3)	para Argumentos de entrada para Avaliação de expressões e Retornos.
\$sp (29 - Pilha), \$ra (31 - Retorno), \$gp (28 - Global), \$at (1)	Controle do Sistema.



Nota: Cada registrador possui exatos 32 bits de largura.

Organização da Memória: Palavras e Alinhamento

A memória MIPS é um grande vetor unidimensional de bytes, mas o processador consome dados em blocos padronizados.



O Projeto Lógico: Segmentação de Código e Dados

A Oficina: .text

- Onde habitam as instruções do programa. O algoritmo em si.
- Ponto de Entrada: Marcado pelo label `main`:

```
add $t0, $s0, $s1
```

O Almoxarifado: .data

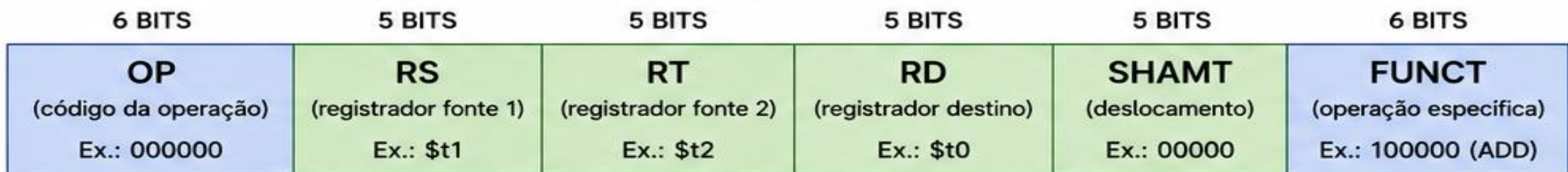
- Declaração de variáveis estáticas, arrays e strings na RAM.
- Característica: Aloca espaço antes do código rodar.

```
array: .space 40  
msg:   .asciiz 'Olá'
```

O caractere '#' atua como barreira de comentários, orientando humanos sem interferir na máquina.

FORMATOS DE INSTRUÇÕES MIPS

31 ————— 32 BITS —————> 0



OP : 000000 → identifica que a instrução é do tipo R	RD : número do registrador destino (0 a 31)
RS : número do registrador fonte 1 (0 a 31)	SHAMT : quantidade de deslocamento para instruções de shift (0 a 31)
RT : número do registrador fonte 2 (0 a 31)	FUNCT : define a operação exata (ex.: 100000=ADD, 100010=SUB, 100100=AND)

EXEMPLO:

`add $t0, $t1, $t2`

OP	=	000000 (tipo R)
RS	=	\$t1 (9)
RT	=	\$t2 (10)
RD	=	\$t0 (8)
SHAMT	=	00000 (não usado)
FUNCT	=	100000 (ADD)

TIPO R
(Registrador)

Usado para operações entre registradores (aritméticas e lógicas).

31 ————— 32 BITS —————> 0



OP : código da operação (ex.: 001000=ADDI, 100011=LW, 000100=BEQ)	IMMEDIATE : valor imediato de 16 bits (constante ou deslocamento/offset)
RS : número do registrador fonte (0 a 31)	
RT : número do registrador destino (0 a 31)	

EXEMPLOS:

`add $t0, $t1, 10`

OP	=	001000 (ADDI)
RS	=	\$t1 (9)
RT	=	\$t0 (8)
IMMEDIATE	=	0000 0000 0000 1010 (10)

`lw $t0, 4($sp)`

OP	=	100011 (LW)
RS	=	\$sp (29)
RT	=	\$t0 (8)
IMMEDIATE	=	0000 0000 0000 0100 (4)

TIPO I
(Imediato)

Usado para operações com constante imediata, acesso à memória e desvios condicionais.

31 ————— 32 BITS —————> 0



OP : código da operação (ex.: 000010=J, 000011=JAL)
ADDRESS : endereço alvo de 26 bits O endereço final é formado assim: { PC[31:28] , ADDRESS , 2'b00 } → deslocamento de 2 bits à esquerda (×4)

EXEMPLO:

`j LOOP`

OP	=	000010 (J)
ADDRESS	=	endereço do rótulo LOOP (26 bits do endereço >> 2)

O processador concatena os 4 bits mais significativos do PC atual e acrescenta 2 bits 00 ao final.

TIPO J
(Jump)

Usado para desvios incondicionais (saltos para endereço).

Trazendo a Arquitetura para a Prática: MARS

A teoria da arquitetura MIPS ganha vida através do MARS (MIPS Assembler and Runtime Simulator). Escrito em Java e multiplataforma, ele simula o comportamento interno do processador.

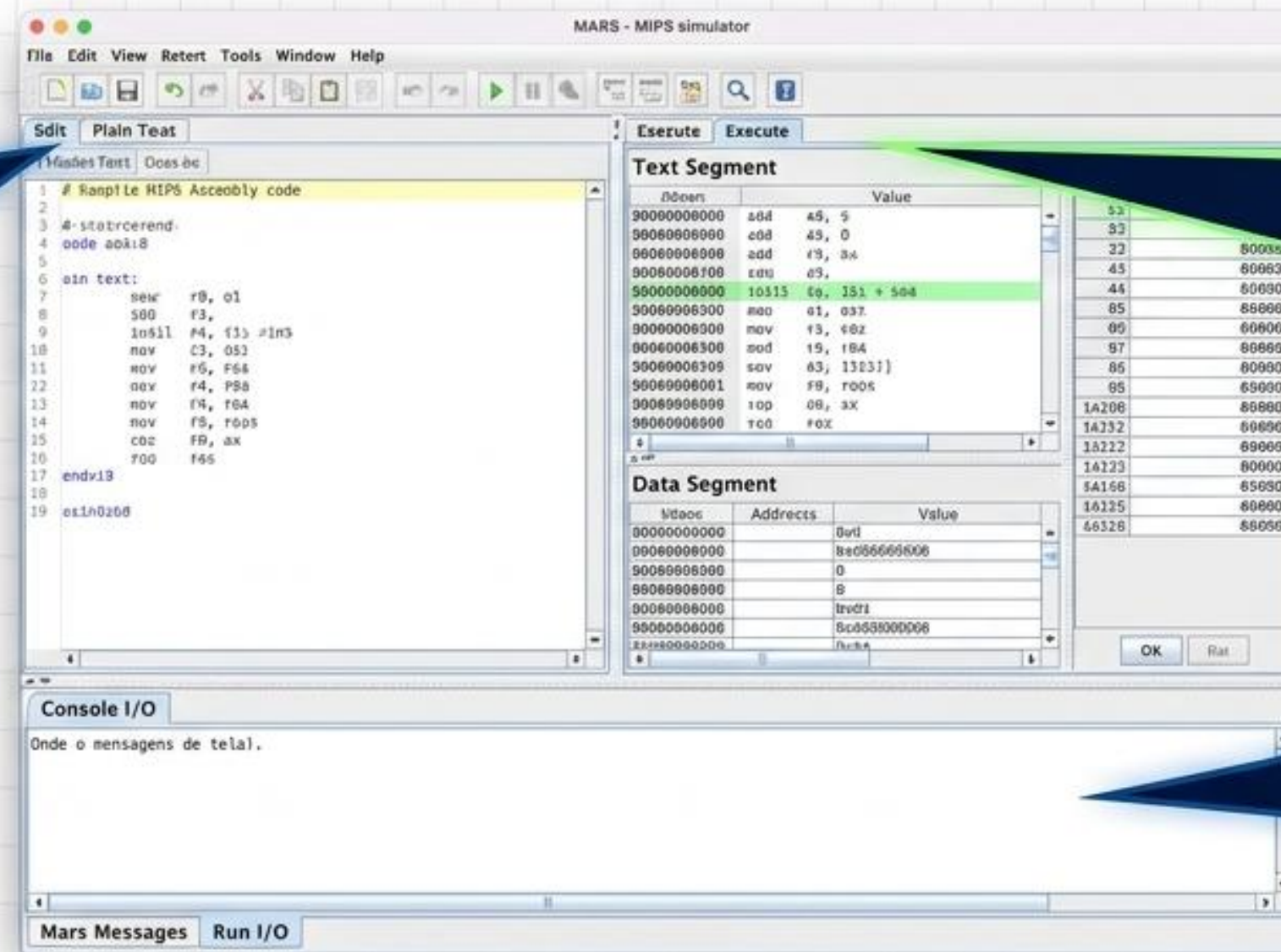


Mais do que um editor, o MARS é um raio-X completo da CPU, permitindo visualizar a propagação de cada bit em tempo real.

Anatomia da Interface: Edição vs. Execução

1

Aba Edit: O painel de texto simples onde o desenvolvedor escreve o código Assembly.



Painéis de Simulação: Visão de Raio-X do Hardware

O Banco de Registradores

MARS Registers		
Name	Number	Value
\$zero	0	0x00000000
\$at	1	0x00000001
\$v0-\$v1	2	0x00000000
\$a0-\$a3	3	0x00000000
\$t0	4	0x00000000
\$t1-\$t7	5	0x00000000
\$t2-\$t3	4	0x00000002
\$t3-\$t4	5	0x00000000
\$s5-\$s5	6	0x14000000
\$s0-\$s7	7	0x00000000
\$t8-\$t9	8	0x00000000
\$k0-\$k1	9	0x00000000
\$gp	9	0x00000000
\$sp	10	0x00000000
\$fp	11	0x00000000
\$ra	12	0x00000000
pc	13231	0x00000000
hi	0	0x00000000
lo	0	0x00000000

Valores piscam em verde quando são ativamente alterados por uma instrução no ciclo atual.

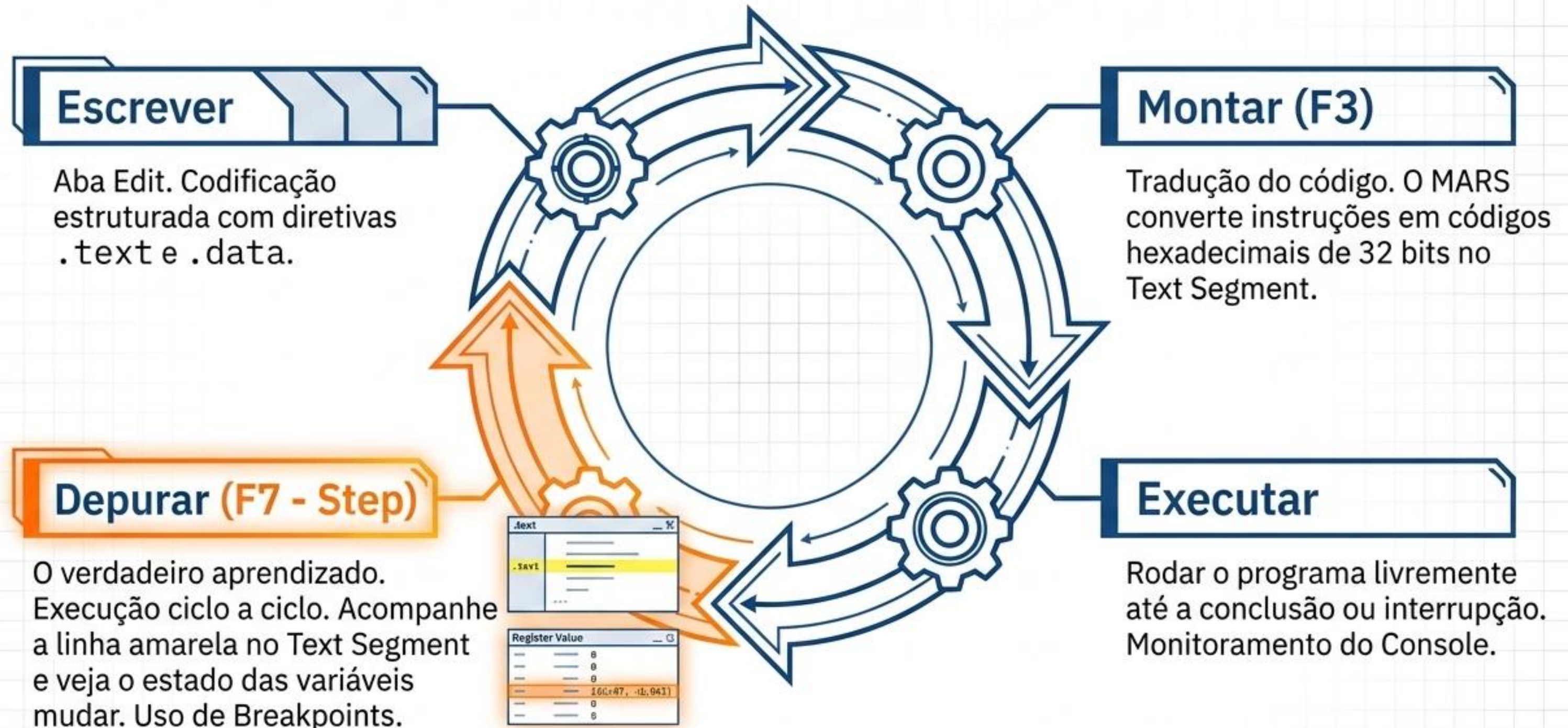
O Segmento de Dados (Memória)

MARS Data Segment			
0x10010000	0x10010004	0x10010008	0x10010112
0x10010004	0x00001100	0x000013EB	0x00001138
0x10010008	0x00001098	0x00001200	0x00001000
0x10010012	0x000020E4	0x000068CE	0x00001E00
0x10010106	0x00005000	0x00000000	0x00000F00
0x10010100	0x00000000	0x00003063	0x00009591
0x10010114	0x00009000	0x00000000	0x00000000
0x10010108	0x00009000	0x00000000	0x00000000
0x10010112	0x000090E0	0x00003E00	0x00002000
0x10010116	0x00001881	0x0000316E	0x00004F00
0x10010130	0x00007800	0x00002F00	0x00007E00
0x10010188	0x00000000	0x00002500	0x00002500
0x10010192	0x00001E00	0x00002500	0x00002301
0x10010204	0x00003000	0x00002500	0x00000000
0x10010206	0x00009000	0x00000000	0x00009000
0x10010228	0x00007800	0x00002500	0x00001010
0x10010230	0x00007800	0x00003000	0x00009000

☒ Hexadecimal Addresses ☒ Hexadecimal Values ☐ ASCII

Acompanhe as variáveis declaradas na RAM em tempo real. Checkboxes na parte inferior permitem alternar entre visualização Hexadecimal e ASCII.

O Ciclo de Desenvolvimento Iterativo



Interação com o Sistema Externo (Syscalls)

O processador MIPS não controla diretamente o monitor ou teclado. Para entrada e saída, ele faz uma “Chamada de Sistema” delegando a ação ao ambiente MARS.

Passo 1: Carregar o código da operação no registrador \$v0.

```
li $v0, [código]
```

Passo 2: Carregar os argumentos (se houver) em \$a0 ou \$a1.

```
la $a0, [endereço] ou li $a0, [valor]
```

Passo 3: Emitir a instrução mágica syscall.

```
syscall
```

Guia de Syscalls \$v0

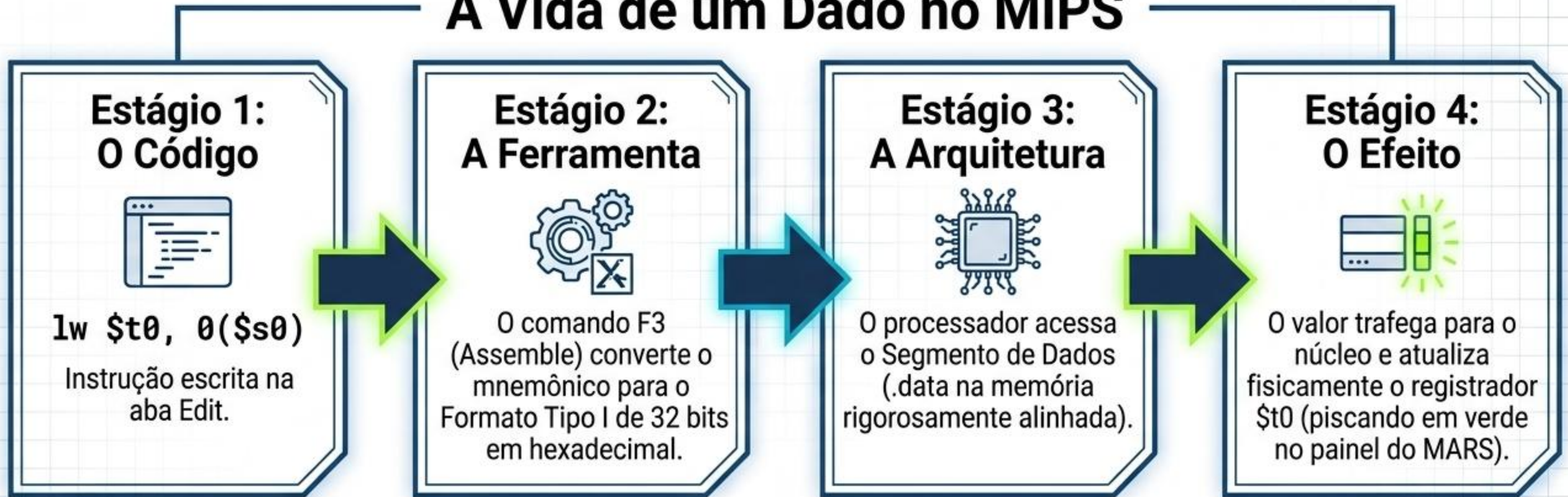
\$v0 = 4 : **Imprimir String**
(Endereço em \$a0)

\$v0 = 5 : **Ler Inteiro**
(Resultado salvo em \$v0)

\$v0 = 10 : **Encerrar Programa**

Síntese: Da Instrução ao Silício Simulado

A Vida de um Dado no MIPS



**A simulação em MARS não é apenas escrever código;
é projetar o fluxo físico da informação através da arquitetura.**

Materiais e atividades:

https://github.com/danilomonteirosoouza/mars_simulator/tree/main/Aula%202

DISCIPLINA
HOJE
LIBERDADE
AMANHÃ.

TALENTO
NÃO É NADA
SEM TRABALHO
DURO.

VAMOS A PRÁTICA?

FOCO

- DISCIPLINA
- CONSISTÊNCIA
- ORGANIZAÇÃO
- PERSISTÊNCIA

= RESULTADOS ★

PLANO:

- ☐ ESTUDAR
- ☐ REVISAR
- ☐ PRATICAR
- ☐ DESCANSAR
- ☐ REPETIR

VOCÊ É
SEU ÚNICO
LIMITE.

19:30

FOCO | DISCIPLINA | CONSTÂNCIA

DISCIPLINA
FOCO
EXECUÇÃO

FOCO
NO
OBJETIVO.

MENTE
FORTE
CORPO
FORTE
MISSÃO
CLARA.

RESUMO

Conceitos importantes

- Entender o básico
- Fazer ligações
- Praticar exercícios
- Revisar sempre
- Ensinar o que aprendeu

PEQUENOS PASSOS
TODOS OS DIAS.
GRANDES RESULTADOS.

EXERCÍCIOS

1 $f(x) = ax^2 + bx + c$
 $\Delta = b^2 - 4ac$
 $\Delta = 0 \quad \uparrow \quad 2 \text{ raízes reais}$
 $\Delta = 0 \quad \downarrow \quad 1 \text{ raiz real}$
 $\Delta < 0 \quad \downarrow \quad 2 \text{ raízes complexas}$

2 $f(x) = x^2 - 4$
 $x^2 = 4$
 $x = \pm 2$

3 $f(x) = 2x + 1$
 $f(1) = 2(1) + 1 = 3$
 $f(2) = 2(2) + 1 = 5$

PRÁTICA
GERA
CONFIANÇA.

PLANO DE ESTUDO

- ☐ MATEMÁTICA
- ☐ FÍSICA
- ☐ QUÍMICA
- ☐ REDAÇÃO
- ☐ REVISÃO GERAL